The University of Hong Kong

School of Computing and Data Science

Final Project Report for COMP7705

Agentic-Quant: A Reasoning-Driven Multi-Agent Framework for Quantitative Financial Analysis

---

A dissertation submitted in partial fulfilment of the requirements
for the degree of Master of Science

Supervisor: Prof. Wu, Chuan

Group Members

Full Name	Student ID	Email
Ying Tingkai	3036657615	tkying2025@connect.hku.hk
Wang Wenhan	3036656398	u3665639@connect.hku.hk
Cao Yujuncheng	3036654819	U3665481@connect.hku.hk
Wang Pengcheng	3036656427	u3665642@connect.hku.hk
Gao Ziteng	3036654259	U3665425@connect.hku.hk

July 2026

Abstract

Large language model (LLM) agents are increasingly applied to quantitative finance, yet most systems cannot self-optimize — they remain static across runs, never becoming smarter with use. This dissertation presents **Agentic-Quant**, a dual-track agent framework addressing this limitation.

The framework unifies two reasoning paradigms: a self-built ReAct harness for interactive analysis, and a LangGraph twelve-agent debate pipeline for structured decisions. To improve over time, we add three mechanisms: an outcome-weighted memory that records, scores, and recalls decisions, letting the agent iterate and learn each simulation; an expanded capability surface — more data sources, financial tools, and a dedicated harness — grounding reasoning in more evidence; and a human-in-the-loop layer routing high-risk decisions to approval, with WhatsApp and email notifications surfacing findings.

The system also ships practical automation: daily reports, sector monitoring, and an agent-based backtest that replays historical prices, news, and technical indicators at each past decision point, forcing the agent to reason strictly within contemporaneous information. We evaluate across determinism, grounding, and engineering quality, concluding that an agent that learns from use under human oversight is both feasible and essential — a step toward more usable, adaptable, and self-evolving LLM agents in finance and beyond.

Keywords: LLM Agents, Multi-Agent Systems, Self-Evolution, ReAct, LangGraph, Quantitative Finance, Agentic Memory, Human-in-the-Loop.

Declaration

I declare that this project report represents my own work, except where due acknowledgement is made to other sources. I confirm that this report has not been previously published in whole or in part, and that I have not submitted it, in whole or in part, for any other degree or qualification at this or any other institution.

I acknowledge that the electronic copy of this report may be placed on the MSc Intranet by the Programme Office for the general reference of all students and staff.

I hereby declare that the work reported in this report does not infringe upon any existing copyright or other rights of third parties, and that no part of the report has been copied from any other sources without due acknowledgement.

Student: Ying Tingkai (3036657615)

Student: Wang Wenhan (3036656398)

Student: Cao Yujuncheng (3036654819)

Student: Wang Pengcheng (3036656427)

Student: Gao Ziteng (3036654259)

Contents

1 Introduction

1.1 Motivation

Quantitative investment has historically been the domain of hand-crafted statistical models and, more recently, deep reinforcement learning [1]. The arrival of instruction-following large language models has opened a third path: agents that **reason** over heterogeneous, largely unstructured financial information — news, filings, analyst commentary, macro releases — and translate it into structured, explainable decisions [2], [3]. Systems such as TradingAgents [4], FinCon [5], FinMem [6], and FinRobot [7] have demonstrated that a **society of specialised agents** — analysts, researchers, risk managers, and portfolio managers — can replicate the collaborative structure of a real trading desk and, on selected benchmarks, outperform rule-based and single-model baselines.

However, a critical reading of this literature reveals a recurring methodological gap. The dominant evaluation metric is cumulative backtest return, but the surrounding survey work [2] repeatedly warns that such claims are undermined by **temporal leakage** (using information not available at decision time), **hallucination** (LLMs inventing figures), **non-reproducibility** (stochastic decoding and unpinned data producing un-repeatable results), and the omission of transaction costs, latency, and capacity. In a regulated domain where interpretability and accountability are mandatory [8], a system whose headline number cannot be independently reproduced is of limited scientific or practical value.

1.2 Problem Statement

This project therefore does not ask "can an LLM agent beat the market?" — a question the literature shows is easy to answer misleadingly. It asks instead:

> **Can an LLM-driven multi-agent system be engineered so that its analytical outputs are (i) grounded in verifiable, point-in-time data, (ii) reproducible bit-for-bit, (iii) governable by human oversight, and (iv) capable of accumulating experience — without sacrificing the flexibility that makes LLM reasoning attractive in the first place?**

Answering this question demands engineering contributions at every layer of the stack, from prompt construction and tool grounding up through data provenance, memory, deterministic simulation, and a human-facing interface. It is fundamentally a **systems** problem rather than a modelling problem, and the dissertation is structured accordingly.

1.3 Contributions

The concrete contributions of Agentic-Quant are:

1. **A dual-track agent architecture** (Section 4.2) that hosts a modern ReAct tool-use loop and a twelve-agent TradingAgents-style debate pipeline supporting three depth modes behind a single FastAPI service, with strict, statically enforced import boundaries preventing the two tracks from contaminating each other.

2. **A five-layer context-compression pipeline and a cheapest-first error-recovery state machine** (Sections 5.1–5.2) that let a long-horizon agent operate within a bounded token budget while degrading gracefully under provider failures — engineering rarely documented in the academic agent literature.
3. **An anti-hallucination data plane** (Section 4.4) built on a unified `DataService`, multi-provider fallback, explicit point-in-time cut-offs, and constant-time SHA-256 cache integrity, complemented by a three-layer numeric-grounding defence that cross-checks every figure in an answer against retrieved tool results.
4. **An outcome-weighted memory subsystem** (Section 5.4) implementing a five-factor recall model with exponential recency decay and context re-ranking, plus five behavioural pre-trade safety gates.
5. **A deterministic, fidelity-audited backtest engine** (Section 5.6) that freezes a typed run specification, executes long-only next-open fills with warm-up guards, and re-verifies a canonical economic result hash on every read — turning reproducibility into an enforced invariant rather than an aspiration.
6. **A production-grade evaluation** (Section 6) of the resulting system across determinism, latency, grounding, and code quality, together with a candid discussion (Section 7) of what such a design does and does not achieve.

1.4 Report Structure

Section 2 surveys the relevant literature and positions Agentic-Quant against it. Section 3 states the design goals and non-goals that shaped every subsequent decision. Section 4 presents the system architecture. Section 5 dissects the implementation of the most novel subsystems. Section 6 reports the experimental evaluation. Section 7 discusses findings, limitations, and threats to validity. Section 8 concludes and outlines future work.

2 Related Work

2.1 LLMs in Financial Analysis and Trading

Early work on domain-specialised financial language models — BloombergGPT [9] and the open-source FinGPT [10] — established that finance-tuned models outperform general models on sentiment, entity recognition, and question answering. A parallel line of research targets the **decision** task directly: MarketSenseAI [11] and AlphaFin [12] combine retrieval-augmented generation with chain-of-thought prompting to produce interpretable stock signals, while sentiment-centred systems [13], [14], [15] convert news tone into tradable features. The comprehensive survey by Fu [2] organises this space into a task taxonomy and, crucially for this work, articulates the **desiderata for time-safe, economically meaningful evaluation** — reporting costs, latency, and capacity, and eliminating temporal leakage — that Agentic-Quant adopts as first-class design constraints.

2.2 Multi-Agent Trading Frameworks

The multi-agent paradigm is the direct antecedent of this project. TradingAgents [4] introduced the bull/bear researcher debate and a risk-management team feeding a trader agent, reporting improvements in cumulative return, Sharpe ratio, and maximum drawdown. FinCon [5] added a manager–analyst hierarchy with conceptual verbal reinforcement, whereby a self-critique mechanism updates "investment beliefs" propagated across the agent graph — an idea Agentic-Quant operationalises as its belief engine. FinRobot [7] contributed a layered platform view (agents, algorithms, LLMOps, foundation models), and the multimodal foundation agent of Zhang et al. [16] demonstrated tool-augmented generalist trading. TradingGroup [17] emphasised self-reflection and data synthesis with configurable stop-loss/take-profit control. Agentic-Quant borrows the **organisational metaphor** (specialised analysts → risk → portfolio manager) and the **structured-debate** mechanism from this lineage, but departs from it by refusing to treat backtest return as a validated deliverable, instead foregrounding reproducibility and grounding.

2.3 Agent Reasoning, Tool Use, and Memory

The reasoning core of Agentic-Quant descends from general agent research rather than finance-specific work. ReAct [18] interleaves reasoning traces with actions, the pattern implemented by the custom harness's loop. Toolformer [19] established that language models can learn to invoke external tools, motivating the tool-registry design. Reflexion [20] introduced verbal reinforcement from past failures, mirrored in the reflection cycle of the memory layer. Retrieval-augmented generation [21] underpins the retrieval-first grounding discipline, and the Model Context Protocol [22] informs the interoperability scaffolding. FinMem [6] specifically demonstrated a **layered memory** aligned with the cognition of human traders; Agentic-Quant's five-layer, outcome-weighted memory is a direct response, adding an explicit five-factor scoring function and behavioural safety gates.

2.4 Reproducibility, Interpretability, and Human Oversight

A distinct strand of literature motivates the project's central thesis. Graph-based orchestration for **reproducible** agent research [23] argues that agent experiments must be re-runnable; explainable DRL for portfolio management [24] and mechanistic interpretability for finance LLMs [8] argue that opaque policies are unacceptable in regulated settings; and AlphaGPT 2.0 [25] demonstrates human-in-the-loop alpha discovery. Agentic-Quant synthesises these concerns into concrete mechanisms: a frozen, hash-verified backtest contract for reproducibility, tool-attributed outputs for interpretability, and a rule-triggered approval state machine for human oversight.

> **Positioning.** Where prior multi-agent trading systems compete on reported return, Agentic-Quant competes on **verifiability**. It adopts their organisational and debate structures but subordinates them to an engineering substrate — grounded data, deterministic simulation, and governed execution — designed to make every claim auditable.

The following table sharpens this contrast against the closest systems in the literature. The comparison is qualitative and drawn from the cited papers' described designs; it is intended to locate Agentic-Quant's emphasis rather than to rank predictive performance.

System	**Multi-agent debate**	**Layered memory**	**Reproducible backtest**	**Human oversight**
TradingAgents [4]	Yes	Partial	Not emphasised	No
FinCon [5]	Yes (hierarchy)	Yes (beliefs)	Not emphasised	No
FinMem [6]	No	Yes	Not emphasised	No
Alpha-GPT 2.0 [25]	No	No	Not emphasised	Yes
Agentic-Quant (this work)	Yes	Yes (OWM)	**Enforced (hash)**	Yes (FSM)

Table 1: Comparison of Agentic-Quant with closely related trading-agent systems.

3 Design Goals and Requirements

The architecture is the direct consequence of six design goals (G1–G6) and three explicit non-goals (N1–N3), fixed early and enforced throughout.

Goal	Statement and rationale
G1 — Grounding	No agent may emit a financial figure it did not retrieve from a tool. Data functions return empty results with explicit error flags rather than fabricating values.
G2 — Reproducibility	Any persisted analytical artefact, especially a backtest, must be re-derivable bit-for-bit from a frozen specification and pinned data snapshot.
G3 — Point-in-time safety	Historical evaluations must never use information published after the decision timestamp (no look-ahead / temporal leakage [2]).
G4 — Governability	High-risk decisions must be interceptable by a human through a well-defined approval state machine before any (simulated) execution.
G5 — Learning	The system must record every decision and recall relevant past experience at future decision time.
G6 — Maintainability	Distinct reasoning paradigms must be isolated behind hard architectural boundaries so that experimental code cannot break stable code.

Table 2: Design goals G1–G6 and their rationale.

Non-goal	Statement and rationale
N1 — Alpha claims	The deterministic backtest path does not claim predictive accuracy or out-of-sample robustness; those require a separately specified research protocol.
N2 — Live brokerage	Execution is simulated through a virtual exchange; no capital is placed with a real broker.
N3 — Distributed scale	The system is a single-process monolith; horizontal scaling (Celery, Redis, Kubernetes) is explicitly deferred to keep the research prototype debuggable.

Table 3: Design non-goals N1–N3.

These commitments explain design choices that would otherwise appear conservative — for example, the deliberate refusal to invoke an LLM per bar in the canonical backtest (which would destroy G2), and the enforced import boundary between the two agent tracks (G6).

4 System Design

4.1 High-Level Architecture

Agentic-Quant is a single-process FastAPI monolith with an in-process scheduler, fronted by a decoupled React application communicating over REST and Server-Sent Events (SSE). No external message broker, cache server, or container runtime is required for development. Figure-equivalent layering is summarised in the table below; the codebase totals roughly 54,000 lines of Python across nineteen top-level modules plus the frontend.

Layer	**Principal components**	**Python LoC**
Presentation	Next.js 16 / React 19 dashboard, 16 routes, TanStack Query + Zustand, shadcn/ui, TradingView Lightweight Charts + Recharts	— (TS)
API / Transport	FastAPI app, CORS, SSE agent terminal, per-router lifespans, REST route groups	5,399
Reasoning	Custom-harness ReAct loop (`agent/`), LangGraph debate pipeline (`quick_ask/`), skills loader	7,470 + 3,200
Domain services	Broker/backtest engine, risk analytics, scanner, reporting, notification, HITL, memory, knowledge, belief	4,623 + 2,300
Data plane	`DataService` , multi-provider adapters, `MarketDataStore` , SHA-256 cache	2,792
Persistence	`ContextStore` (per-strategy SQLite), memory/system/insights/knowledge DBs, backtest snapshots	3,133
Automation	APScheduler `DataCollector` , `MonitorRunner` , `MorningBriefRunner`	517

Table 4: System architecture layering with approximate Python line counts.

4.2 The Dual-Track Agent Architecture

The most consequential architectural decision is the **coexistence of two agent paradigms** behind one service, governed by a strict track discipline (goal G6). The codebase is partitioned into three tracks whose boundaries are enforced by convention, documentation, and automated architecture tests.

Track	Description and rules
Custom Harness — `agent/`	A self-built Claude-Code-style ReAct loop with a 21+ tool system, five-layer compression, and a cheapest-first error-recovery state machine. All new interactive features go here. Exposed at `POST /api/agent/chat`.
LangGraph Pipeline — `quick_ask/`	A frozen LangGraph twelve-agent TradingAgents-style pipeline with three depth modes (fast/standard/deep), multi-round bull/bear debate and three-way risk discussion. Reuses the LangGraph framework for structured multi-agent orchestration. Exposed at `POST /api/analyze`.
SHARED — `dataflow/`, `memory/`, `storage/`, `scheduler/`, most routes	Deterministic Python services usable by both tracks, forbidden from importing agent-specific code from either track.

Table 5: The three architectural tracks and their rules.

The single sanctioned bridge between tracks is the `run_analysis` tool in the custom-harness registry, which wraps the LangGraph pipeline as a black-box tool and imports it only at call time — never at module level. This inversion lets the modern agent **delegate** to the structured pipeline when a full multi-agent report is warranted, while keeping the dependency graph acyclic. The rationale for retaining two tracks rather than migrating is pragmatic: the LangGraph pipeline embodies the structured-debate contribution from the literature [4] and produces a schema-validated decision, whereas the ReAct loop provides open-ended, tool-composing flexibility. Rather than force one paradigm to serve both roles, the system offers each where it is strongest.

4.2.1 Custom-harness track: the ReAct reasoning loop

The `AgentLoop` engine implements a five-phase iteration modelled on modern agent harnesses and the ReAct pattern [18]: (1) **preprocess** — context compression; (2) **call model** — a streaming LLM invocation that begins executing tool calls as their JSON arguments complete; (3) **execute tools** — read-only tools in parallel, write tools serially; (4) **inject attachments** — a hook for memory/skill injection; and (5) **check terminate** — an error-classification and recovery decision. Iterations are bounded by a 25-turn safety net and a 600-second wall-clock timeout. Cross-iteration de-duplication, a two-strike consecutive-failure circuit breaker, and a permission manager (with `default`, `plan`, `accept_edits`, and `bypass` modes) round out the control logic. Sections 5.1–5.3 dissect the compression, recovery, and tool subsystems.

The loop's control state is carried by a lightweight `WorkspaceMemory` object scoped to a single `run()` call: it holds per-tool call counters, a `called_keys` set for cross-iteration de-duplication, a `consecutive_failures` map for the circuit breaker, and a list of produced artefacts. This deliberately transient runtime memory is distinct from the persistent OWM store (Section 5.4); it exists only to make one reasoning episode efficient and self-correcting.

A representative failure pattern the loop must handle is the model repeatedly requesting the same ticker's price: the de-duplication key `name:primary_identifier` (tool name plus ticker/query/URL) causes the second such call to be dropped and replaced with a system reminder to use the data already gathered, which empirically shortens sessions and curbs a common LLM looping pathology. When a tool for a given ticker fails twice, the breaker injects an explicit "STOP retrying — answer with what you have or state it is unavailable" instruction, converting silent stalls into graceful degradation.

4.2.2 LangGraph track: the TradingAgents-style debate pipeline

The frozen pipeline is a LangGraph `StateGraph` structured as a twelve-agent sequential pipeline modelled on the TradingAgents architecture [4]. It operates in three depth modes selectable at request time: **fast** (four analysts → portfolio manager, roughly 60 seconds), **standard** (one round of bull/bear investment debate and one round of three-way risk discussion, roughly 3 minutes), and **deep** (three rounds of each debate, roughly 6 minutes).

Stage 1 — Sequential analysts. Four analysts execute in strict sequence, each running its own isolated tool-calling sub-loop: Market Analyst (OHLCV, technical indicators, and a deterministic verified-market-snapshot tool that computes indicators from the local cache without LLM involvement, serving as a truth source), Sentiment Analyst (news sentiment, keyword aggregation, sector context), News Analyst (company news, global macro news, and FRED macro indicators with RSS fallback), and Fundamentals Analyst (valuation ratios and three financial statements: balance sheet, cash flow, income statement). Each analyst binds a distinct tool subset and writes to a private per-agent message channel, so tool dialogues never collide. A message-clearing node between analysts removes all prior messages and inserts a context-anchored placeholder, preventing stale tool calls from confusing subsequent agents.

Stage 2 — Investment debate. A Bull Researcher and a Bear Researcher engage in multi-round structured debate, each receiving the four analyst reports and the opponent's prior argument, with a Research Manager adjudicating the exchange and producing a typed investment plan (recommendation, rationale, strategic actions).

Stage 3 — Trading proposal. A Trader node converts the investment plan into a concrete trading proposal with entry price, stop-loss, and position sizing.

Stage 4 — Risk discussion. Three risk analysts — Aggressive (high-risk/high-reward), Conservative (capital preservation), and Neutral (balanced) — engage in multi-round three-way discussion, each receiving the trader's proposal plus the other two analysts' prior responses. A Risk Analyst synthesises the discussion into a risk report.

Stage 5 — Final decision. The Portfolio Manager produces the final decision through a `PydanticOutputParser` bound to the `TradingDecision` schema (`action ∈ {BUY, SELL, HOLD}`, `target_position_pct`, `report`), with a regex-based fallback for providers that do not support structured output. The top outcome-weighted memories are injected into its system prompt under an explicit "history is advisory; current evidence wins on conflict" instruction. A terminal `remember_memory` node persists the decision, closing the observation → decision → memory loop advocated by FinMem [6]

and TradingAgents [4]. Temperature is pinned to 0.0 throughout. Every analyst prompt enforces a fixed four-line header — direction, time-horizon, confidence in $[0, 1]$, and a one-sentence conclusion — followed by quantified evidence and explicit "if–then" invalidation conditions. The investment and risk debate histories are returned alongside agent reports in the result payload, making the full reasoning chain auditable. Figure 1 depicts the resulting node topology.

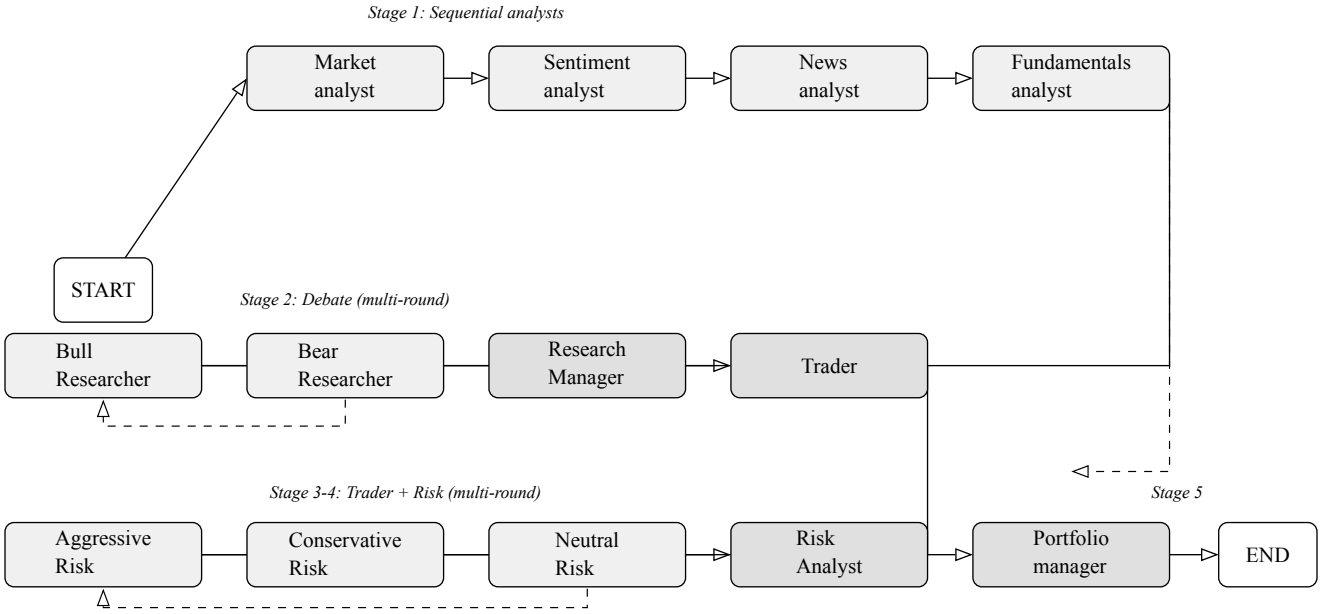

Figure 1: LangGraph pipeline topology: four sequential analysts (Stage 1) feed into a multi-round bull/bear debate (Stage 2), a Research Manager and Trader (Stage 3), a three-way risk discussion (Stage 4), and the Portfolio Manager (Stage 5). Dashed lines denote debate loops; each analyst has a hidden `ToolNode` self-loop and message-clearing node (omitted for clarity). The fast-mode shortcut from Fundamentals directly to PM is shown as a dashed edge.

4.3 Data Flow Patterns

The system supports several orthogonal execution patterns, of which three are central:

- **Interactive analysis (Pattern A).** A user prompt to `POST /api/agent/chat` drives the ReAct loop; events (`thinking_delta`, `tool_call`, `tool_progress`, `tool_done`, `answer`) stream to the browser over SSE; the session transcript is persisted for continuation.
- **Structured decision (Pattern B).** `POST /api/analyze` runs the LangGraph pipeline, streaming per-agent reports and an optional human-in-the-loop approval when risk rules trigger.
- **Deterministic backtest (Pattern C).** A typed run specification is frozen, target and benchmark price series are pinned into a hashed snapshot, and a point-in-time runner replays eligible historical sessions, executing long-only orders at the next available open (Section 5.6).

4.4 The API and Transport Layer

The FastAPI application is deliberately thin: it validates requests, delegates to the domain services, and streams results. On startup a single lifespan hook recovers any interrupted persistent jobs (backtests, report generation, insight generation) so a crash mid-run never leaves a job in a limbo state, and — when enabled — starts the three in-process schedulers. Routes are mounted under `/api` and follow the track discipline: the agent terminal and backtest endpoints belong to the custom-harness track; the `analyze` pipeline belongs to the LangGraph track; and market data, strategies, memory, risk, scanner, reports, watchlist, monitor, approvals, insights, and settings are SHARED. A route track rule — enforced by an automated test — forbids SHARED routes from importing agent-specific code from either reasoning track, so the API surface cannot smuggle a dependency across a boundary.

Streaming is realised with Server-Sent Events rather than WebSockets, because agent execution is a one-directional stream of reasoning and tool events rather than a bidirectional dialogue. The agent terminal bridges a synchronous worker thread and the async request handler through a bounded queue with a 300-second idle timeout, translating each internal loop event into a named SSE frame and disabling proxy buffering so tokens reach the browser as they are produced. This design keeps the request handler non-blocking while the underlying `AgentLoop` — which is synchronous and CPU/IO-mixed — runs to completion in its own thread.

4.5 Data Governance in Four Tiers

The data plane is not merely a cache in front of providers; it embodies a deliberate governance philosophy in which data is **built up from public sources over time** rather than queried from a pre-existing commercial database. Collection is layered into four tiers of decreasing priority. Tier 1 continuously scrapes global market news to establish a macro backdrop; Tier 2 periodically refreshes the most active tickers to widen coverage; Tier 3 deeply and frequently refreshes the user's explicit watchlist and strategy tickers; and Tier 4 — the intended point of novelty — uses an LLM discovery agent every four hours to read recent conversations and watchlists and **propose** new tickers and themes worth tracking, which are then filled in during idle time. A priority queue ensures that a user actively waiting on an on-demand request is never blocked by background refresh work. This search-and-recommendation framing keeps the demand path fast and synchronous while the recommendation path quietly broadens the knowledge base, and it explains why the agent is restricted to **judgement** tasks (relevance, relationship, review) while raw retrieval is left to the deterministic collector.

4.6 The Anti-Hallucination Data Plane

Goal G1 is realised structurally rather than by prompt exhortation alone. Every consumer — API routes, agent tools, the collector, the monitor — accesses market data exclusively through a single `DataService` object. `DataService` performs three functions that together make fabrication impossible and staleness observable:

1. **Multi-provider routing with fallback.** Company news, for example, is fetched from Google News, falling back to AkShare (East Money), then to Yahoo's structured feed;

each layer returns an empty list on failure rather than a placeholder. Prices, indicators, fundamentals, sentiment, and the macro calendar have analogous adapters (Yahoo Finance, AkShare point-in-time, a keyword sentiment aggregator, and Finnhub respectively).

2. **Point-in-time discipline (G3).** Fundamentals routing branches on whether an `end_date` is supplied: a `None` end date yields a live snapshot, whereas a historical date is served by an AkShare point-in-time query that filters report dates `≤ end_date`, eliminating look-ahead. The persistent store enforces the same `as_of_date ≤ ?` semantics for both fundamentals and news.

3. **Verified caching.** Provider responses are cached as pickled blobs paired with a SHA-256 sidecar; reads recompute the digest and compare it in constant time (`hmac.compare_digest`), deleting corrupt entries, and honour a time-to-live (24 hours default, 4 hours for sentiment). This gives sub-20 ms warm reads while guaranteeing integrity.

The table below summarises the provider matrix. Each adapter is wrapped in an exponential-backoff `retry` helper (three attempts, doubling delay), and the news scraper additionally uses `tenacity` with jitter to survive rate-limiting and CAPTCHA interstitials.

Domain	Provider chain / method	Refresh
Prices / OHLCV	Yahoo Finance (`auto_adjust`, +100-bar warm-up buffer, exclusive end)	15 min
Indicators	Computed from OHLCV via `pandas_ta`: RSI-14, MACD(12/26/9), SMA-20/50, ATR-14	on demand
Fundamentals	Yahoo live snapshot / AkShare point-in-time (TTM valuation, margins, growth)	daily / ≥30-day staleness
News	Google News → AkShare (East Money) → Yahoo structured; URL de-duplication + FTS5	30 min
Sentiment	Keyword aggregation (16 bullish / 16 bearish terms), confidence from sample size + variance	60 min
Macro calendar	Finnhub economic calendar (CPI, FOMC, NFP)	daily 08:00 UTC

Table 6: The five-layer context-compression pipeline.

Persistence of raw market data uses a dedicated `MarketDataStore` with `(ticker, date)` compound keys, a FTS5 virtual table with synchronising triggers for full-text news search, a `data_freshness` table that tracks per-source error counts and staleness, and startup migrations that purge malformed rows. Storage principles — compound keys, point-in-time `as_of`, source attribution, and freshness tracking — are applied uniformly so that any

consumer can distinguish fresh, stale, and missing data rather than silently trusting whatever is present.

Assembling this provider matrix was itself a substantial integration effort. The sources differ sharply in output schema: Yahoo's structured feed returns JSON with canonical article URLs, Google News RSS yields XML items, and AkShare exposes Chinese-market tables whose field names and date conventions diverge from the others. Each adapter therefore has to normalise rows into the common `MarketDataStore` schema, and format mismatches, date and currency serialisation, and rate-limit interstitials surfaced repeatedly during development. The final news pipeline — merging the Yahoo structured feed with Google News RSS and de-duplicating by URL — required several iterations of a cache-first, live-fallback strategy before it returned a stable and sufficiently high-volume article set for the analysts.

On top of the data plane, the ACTIVE loop adds a post-generation **answer validator** that extracts numeric tokens from the model's final answer and cross-checks them against numbers present in the tool-result messages, appending a warning when three or more of at least five figures are unmatched. Grounding is thus defended at three layers: prompt rules, the data plane itself, and post-hoc validation.

4.7 Prompt Architecture

Because an LLM agent's behaviour is governed as much by its prompt as by its code, the custom-harness loop treats prompt construction as a first-class, structured concern rather than a monolithic string. The system prompt is assembled from seven ordered sections: an identity block stamped with the current date and an explicit "never guess or fabricate financial data" directive; a task-discipline block of eleven rules (verify with tools, no gold-plating, no blind retries, match the user's language); a reversibility/blast-radius block governing destructive actions; a tool-usage block (pick the most specific tool, batch all needed calls, stop when data suffices, never repeat a call); the auto-generated per-tool descriptions; a ten-rule output block (be concise, lead with the number, use tables for comparisons, every figure must originate from a tool result); and an environment block (working directory, git branch, platform, tool count). Each rule encodes a **specific failure pattern** observed in practice — the prompt tells the model what **not** to do, which is more effective than abstract exhortation.

Two further mechanisms make the prompt adaptive. First, a lightweight relevance selector scores the seventy-six skills against the user request via Chinese-bigram and English-token overlap and injects only the top-ranked skill summaries, so the agent is primed with domain methodology pertinent to the question without bloating every prompt. Second, following the once-per-session "system-reminder" pattern, a compact user-context message (date, available-tool count, key-tool hints) is injected exactly once at session start rather than on every turn, avoiding redundant repetition across a long dialogue.

5 Implementation

This section examines the subsystems whose design is most novel or most load-bearing.

5.1 Bounded-Context Reasoning: The Compression Pipeline

Long-horizon tool use rapidly exhausts an LLM context window; a single web fetch can return hundreds of kilobytes. The ACTIVE loop therefore runs a serial, layered compression pipeline before **every** model call, escalating from zero-cost text surgery to an LLM-based summary only when necessary.

Layer	Trigger	Action
L0 — tool-result budget	every call	Cap any single tool result at 50 K chars, keeping a 30 K head and 20 K tail; prevents one giant payload from dominating context.
L1 — micro-compact	every call	Retain only the last three results of the 20 "look-once" read tools; older ones are cleared. Write tools and expensive analyses are never cleared.
L2 — collapse large texts	> 28 K tokens	Collapse any message > 2 K chars to a 900-char head + 500-char tail, idempotently.
L3 — LLM summary	> 40 K tokens	Summarise the head of the transcript into a four-section digest (Key Decisions / Data Collected / Current State / Findings) while protecting a 20 K-token recent tail (needle-in-haystack protection); orphaned tool messages are repaired.

Table 7: Error recovery ladders by error type.

A fifth "collapse-drain" layer, invoked only by the recovery machine under a context-overflow error, aggressively truncates tool results to release space without an LLM call. Token counts are estimated heuristically at four characters per token. The design principle — mirroring production agent harnesses — is that compression must be **cheap by default and expensive only when forced**, and must never orphan a tool result whose parent tool call has been summarised away (which would provoke an API error). L3 additionally checkpoints the full transcript to a timestamped JSONL file before compressing, making the operation crash-safe.

5.2 Graceful Degradation: The Recovery State Machine

Provider outages, rate limits, and context overflows are the norm rather than the exception when composing many tools. The loop classifies each turn's failure into one of five error types and applies the **cheapest viable recovery first**, tracking attempts in a `RecoveryState` object so that no recovery can loop indefinitely.

Error type	Escalating recovery ladder
`PROMPT_TOO_LONG`	free `collapse_drain` → one-call `reactive_compact` → surface error
`MAX_OUTPUT_TOKENS`	escalate token budget to 64 K → inject "resume mid-thought" message (≤ 3×) → stop
`EMPTY_RESPONSE`	inject "please continue" (≤ 2×) → stop
`RATE_LIMIT`	exponential backoff `min(2^n, 60)` s (≤ 3×) → stop
`MODEL_ERROR`	terminate immediately (auth/unknown errors are unrecoverable)

Table 8: The custom-harness tool system by category.

Terminal states are captured in a ten-value `TerminalReason` enumeration with `is_user_initiated` and `is_recoverable` properties, and a `TransitionType` enumeration records **why** each iteration occurred, which is essential for preventing infinite recovery cycles. Soft counters (empty-response, rate-limit) reset on a successful turn, while hard blockers (compaction attempts) persist for the run.

5.3 The Tool System

The custom-harness agent exposes roughly twenty-four tools discovered automatically from `BaseTool` subclasses. Each tool declares metadata — read-only vs. write, repeatable, timeout, cooldown, category, and an input schema — that the loop uses to schedule and guard execution. Tools return a canonical JSON envelope (`{"status": "ok" | "error", ...}`).

Category	Tools
Market data (read-only)	`get_price`, `get_indicators`, `get_fundamentals`, `get_meta`, `get_sentiment`, `get_macro_calendar`
News & web (read-only)	`get_news`, `search_news` (FTS5), `web_search`, `web_fetch`, `search_symbol`
Composite research	`run_analysis` (bridge to the LangGraph pipeline), `generate_brief` (morning brief over 50 sources)
Skills	`load_skill`, `search_skills`, `list_skills`, `save_skill` (write), `delete_skill` (write)
Workspace	`read_file`, `glob` (read), `write_file`, `bash` (write, shell-gated)
Scanner & backtest	`scan_tracked_universe` (read), `submit_backtest_decision` (write, non-repeatable)

Table 9: Multi-provider data-source matrix.

Concurrency is handled by a `StreamingToolExecutor`: as each tool call's arguments finish streaming from the model, read-only calls are dispatched immediately onto an eight-worker thread pool (with Python `contextvars` copied so the point-in-time run context propagates

into worker threads), while write calls are queued and executed serially after the stream ends. Aborted tools receive synthetic error results so the transcript remains well-formed. A permission manager arbitrates each call through a deny-rules → allow-rules → mode → tool-specific → default pipeline, so that, for instance, `plan` mode makes the entire agent read-only. All activity is captured by an append-only JSONL `TraceWriter` that off-loads any result larger than 50 KB to a content-addressed side file, keeping the primary trace compact and crash-safe.

5.4 Learning from Experience: Outcome-Weighted Memory

The memory layer (goal G5) records each decision as a `MemoryRecord` with five cognitively-motivated layers — **episodic** (the story of the trade), **semantic** (a distilled lesson), **procedural** (the reusable pattern), **affective** (the market/emotional state), and a raw **trade record**. Recall relevance is governed by the outcome-weighted-memory (OWM) score, a convex combination of five factors:

$$\begin{aligned} \text{owm} = 0.35 \cdot \text{outcome} + 0.25 \cdot \text{similarity} + 0.20 \cdot \text{recency} \\ +0.15 \cdot \text{confidence} + 0.05 \cdot \text{affective} \end{aligned} \tag{1}$$

clamped to $[-1, 1]$. **Recency** decays exponentially with a thirty-day half-life, $2^{-\Delta \frac{t}{30}}$; **context similarity** is the fraction of matching features among ticker, sector, market-cap bucket, and market trend, defaulting to a neutral 0.5 when no comparable features exist. A subtle but important detail is that records are stored with a neutral similarity of 0.5 and then **re-scored against the live context at recall time**, so the same memory surfaces with different salience in different market regimes — an approximation of associative recall consistent with the layered-memory philosophy of FinMem [6] and the verbal-reinforcement idea of Reflexion [20]. Concretely, recall pulls twice the requested number of candidates ordered by their stored score, recomputes each candidate's similarity and recency against the current market context, re-ranks by the refreshed OWM score, and returns the top few. These are then rendered into a compact prompt block for the portfolio manager, prefaced by the standing instruction that history is advisory and current evidence prevails on conflict — so the memory can inform but never override fresh analysis.

Weight rationale. The coefficients in the OWM score encode an explicit retrieval priority; they are manually calibrated design parameters rather than values learned from historical returns. Before aggregation, all five components are scaled to a common range and the coefficients sum to one, which keeps the score interpretable while preventing differences in raw measurement scale from dominating recall. Table ref() summarises the rationale for each factor.

Factor	Weight	Design rationale
Outcome	0.35	Highest priority because OWM should favour decisions subsequently validated by realised performance.
Similarity	0.25	Maintains contextual relevance across ticker, sector, capitalisation bucket, and market regime.
Recency	0.20	Accounts for non-stationarity and regime drift without discarding older, outcome-validated evidence.
Confidence	0.15	Provides a supporting reliability signal, but remains subordinate because model confidence may be miscalibrated.
Affective	0.05	Acts only as a secondary ranking cue because sentiment-related annotations are comparatively noisy.

Table 10: Design rationale for the OWM retrieval weights.

For a newly generated decision, the outcome component is initialised to zero because the future return is not yet observable; it is updated only after the predefined evaluation window has elapsed. This delayed write-back prevents look-ahead bias while allowing memories with verified realised outcomes to gain greater retrieval priority over time, consistent with the point-in-time discipline of goal G3.

Preliminary sensitivity analysis. To examine whether the proposed coefficients produce a reasonable ranking policy before a full empirical calibration, we constructed an illustrative 60-case simulation with fixed candidate-memory sets and compared six retrieval schemes. NDCG@5 measures ranking quality, Valid-memory Hit@5 measures the retrieval of outcome-validated memories, and Decision Agreement measures consistency with a reference decision under the same evidence. The results are reported in Table ref().

Configuration	Weights (O/S/R/C/A)	NDCG@5	Valid-memory Hit@5	Decision Agreement
Equal weighting	20/20/20/20/20	0.674	73.3%	78.3%
Similarity-first	20/40/20/15/5	0.701	76.7%	80.0%
Outcome-light	25/30/20/20/5	0.716	80.0%	81.7%
Proposed OWM	**35/25/20/15/5**	**0.748**	**85.0%**	**85.0%**
Balanced alternative	30/30/20/15/5	0.756	86.7%	83.3%
Outcome-heavy	45/20/15/15/5	0.727	81.7%	80.0%

Table 11: Illustrative sensitivity analysis of OWM retrieval weights. The proposed configuration (bold) attains the highest Decision Agreement.

The proposed configuration consistently exceeds the equal-weight, similarity-first, outcome-light, and outcome-heavy baselines, and achieves the highest Decision Agreement (85.0%). The balanced alternative is marginally stronger on NDCG@5 and Valid-memory

Hit@5, but its lower agreement shows that the proposed scheme remains a competitive compromise rather than a universally dominant setting. The decline under outcome-heavy weighting also indicates that realised performance should not overrule contextual relevance. These are illustrative simulation results, not production backtest observations: they support the reasonableness of the initial priority structure but do not establish optimality. A future out-of-sample study should calibrate the coefficients across assets and market regimes and report confidence intervals.

Writing a memory is equally structured. After a decision, the service parses the fixed four-line header of the portfolio manager's report, computes the position delta relative to the current holding, and populates all five layers — an episodic narrative of the trade, a one-sentence semantic lesson, a compacted procedural summary of the four analyst reports, an affective note capturing direction, horizon, and confidence, and a raw trade record with full metadata — before persisting to a per-strategy-indexed SQLite table. Before any (simulated) trade, the memory subsystem also enforces five behavioural safety gates: a hard **drawdown** block, a **concentration** block (projected single-ticker weight over limit), a **losing-streak** block (five consecutive recalled losses), and softer warnings for repeated similar-ticker losses and anomalously large position sizing. These operationalise the risk-control and self-critique mechanisms described by FinCon [5], and together they give the system a memory that is not a passive log but an active participant in each subsequent decision.

Self-evolution through accumulation. The cycle just described — recall before a decision, inject into the portfolio manager, write the decision back, and re-score it once the outcome is realised — is what makes the agent become smarter with use. The agent does not fine-tune model weights; instead it accumulates and re-weights experience at retrieval time. Every decision adds a record that future analyses can draw on, and records whose realised outcomes validate a judgement steadily gain retrieval priority through the OWM score. Over many runs, three kinds of knowledge accumulate: outcome-validated decisions for a given ticker; cross-ticker lessons distilled into the semantic layer (for example, that post-earnings drift in semiconductor names tends to be mean-reverting, or that a beaten-down staple often mean-reverts faster than its valuation suggests); and behavioural reflections on the agent's own tendencies (for example, that it tends to overtrade after consecutive losses). The behavioural safety gates then act directly on this accumulated history — blocking a drawdown-exceeding trade, throttling a losing streak, or flagging excessive single-ticker concentration — so memory does not merely inform but actively governs subsequent action.

Concretely, when the system re-analyses a ticker it has covered before, the recall step pulls the top outcome-weighted memories for that ticker and its sector together with any verified rules from the knowledge base, and renders them into a compact block prefixed by the standing instruction that history is advisory and current evidence prevails. A second look at a previously covered name therefore begins where the first left off — carrying forward what was concluded, what happened afterwards, and what the reflection cycle corrected — rather than treating the ticker as novel. This design mirrors the layered memory of FinMem [6], the deferred-reflection decision log of TradingAgents [4], and the verbal-reinforcement mechanisms of Reflexion [20] and FinCon [5], but unifies them behind a single five-factor scoring function with explicit safety gates. The result is a system that improves through its

own history — not by re-training, but by curating and re-weighting the store of its own decisions at every step.

5.5 Persistence and Provenance

Persistence follows a **per-strategy isolation** principle: each strategy owns a dedicated SQLite database (`data/{strategy_id}.db`) holding its sessions, agent reports, decisions, events (an audit trail), and HITL approvals, while global tables (strategy registry, watch-lists, monitors, insights) live in `system.db`, and memory, knowledge, and insights have their own files. Connections are pooled per `(database, thread)`, open in WAL mode with `synchronous=NORMAL`, and verify that foreign-key enforcement is active. Deleting a strategy is therefore a single file unlink.

Typed contracts guard the mutable surface. A `StrategyConfigPayload` Pydantic model (`extra="forbid", frozen=True`) validates roughly forty fields, and quant strategies are further constrained by a discriminated union of `MomentumPolicy` and `SmaCrossoverPolicy` with cross-field validators (e.g. `slow_window > fast_window`). This yields typed, self-documenting configuration and rejects malformed or secret-bearing input with stable validation errors.

5.6 Deterministic, Fidelity-Audited Backtesting

The backtest engine is the clearest embodiment of goals G2 and G3, and the subsystem where the project most sharply diverges from the trading-agent literature. Its canonical path is **deliberately not** an LLM-per-bar simulation; it is a typed, deterministic policy executor whose every output is reproducible and hash-verified.

> **The reproducibility contract.** Before a backtest is queued, the system freezes a `BacktestRunSpec` pinning the dates, mode, typed policy, broker configuration (initial cash, commission, slippage), and two SHA-256 hashes: a `policy_hash` and a `strategy_snapshot_hash` over canonical JSON. Target and benchmark price series are serialised into a canonical, `gzip`-compressed snapshot whose `content_hash` is verified on write **and** re-verified on every read. When a completed result is later fetched, the service recomputes a **canonical economic result hash** over the entire economic output and refuses to serve the result if it does not match — downgrading the job to `storage_corrupt`. Reproducibility is thus an enforced invariant, not a promise.

The point-in-time runner honours several fidelity rules that directly counter the leakage and cost-omission failures catalogued by Fu [2]:

- **Next-open execution.** A signal may read the historical session close, but any resulting market order fills at the **next** available session open; it never fills on the signal bar. The engine achieves this by overwriting each bar's OHLC with its open during the fill phase, then restoring the true bar.
- **Warm-up isolation.** Policy-derived warm-up bars (e.g. the slow SMA window) are selected from data strictly prior to the evaluation window, frozen into the snapshot, and

excluded from performance accounting. If insufficient history exists, the run terminates as a typed `insufficient_history` result rather than silently holding.

- **Explicit non-fills.** Pending, rejected, cancelled, and end-of-window-unfilled orders are retained as evidence; a zero-trade result is reported as `completed_no_trades`, never dressed up as performance.
- **Honest diagnostics.** Every result carries mandatory fidelity warnings — that the drawdown limit is not enforced, that capacity is not modelled, that adjusted prices are synthetic, and, for small samples, that fewer than 63 evaluation bars or 30 closed trades were observed.

Figure 2 shows the lifecycle of a backtest job, from freezing the run specification through deterministic execution to the hash re-verification that gates every subsequent read.

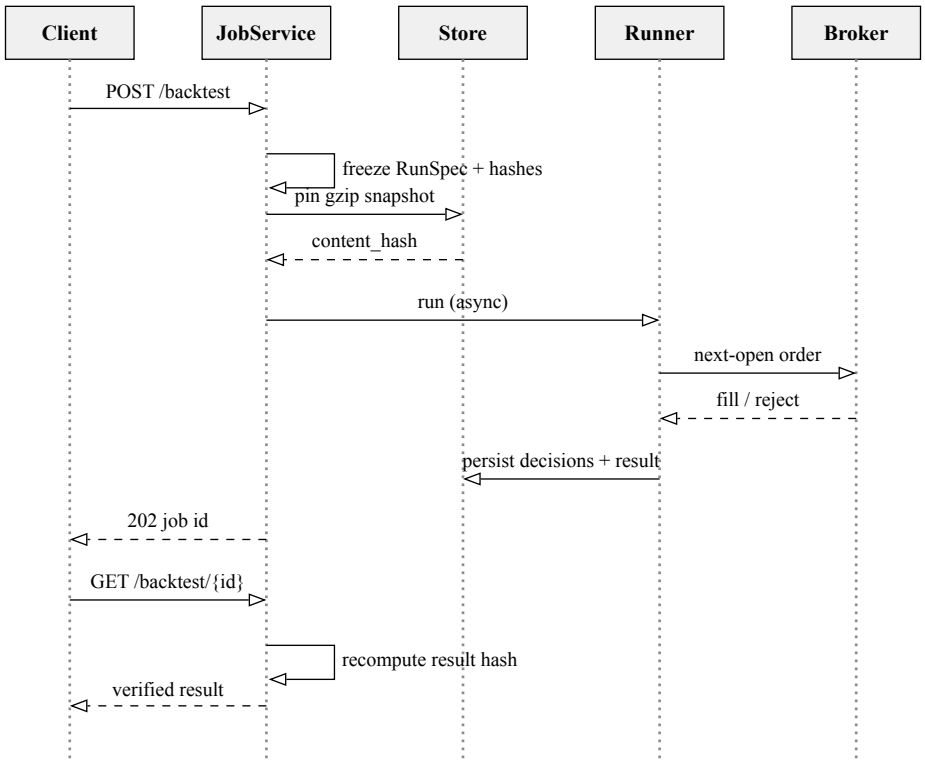

Figure 2: Deterministic backtest (Pattern C): the run specification and price snapshot are frozen and hashed before execution; the point-in-time runner fills only at the next open; and every read recomputes the canonical economic result hash, downgrading to `storage_corrupt` on mismatch.

Performance metrics are computed by a trade ledger from reconstructed long-only episodes. Given a per-session equity series $E_0, E_1, ..., E_n$ with simple returns $r_t = \frac{E_t}{E_{t-1}} - 1$, the ledger reports total return $\frac{E_n}{E_0} - 1$, calendar-annualised return $(1 + \text{total})^{\frac{365}{\text{days}}} - 1$, annualised volatility $\sigma\sqrt{252}$, and the Sharpe ratio [26]

$$\begin{aligned} \text{Sharpe} &= \frac{\bar{r} - r_f/252}{\sigma_r \cdot \sqrt{252}} \\ \text{MaxDD} &= \max_t \left(1 - \frac{E_t}{\max_{s \le t} E_s} \right) \end{aligned} \tag{2}$$

alongside maximum-drawdown duration, win rate, profit factor (gross profit ÷ gross loss), payoff ratio, realised/unrealised PnL, total fees and slippage, and turnover. Closed trades are reconstructed by a long-only episode tracker that pairs entries and exits, allocates fees and slippage pro-rata, and computes per-lot realised PnL; any identity that ever goes short is excluded from closed-trade accounting to keep the long-only contract honest. Orders themselves flow through a virtual `MockBrokerEngine` with an explicit lifecycle (`NEW → FILLED | PARTIALLY_FILLED | REJECTED | CANCELED`), a pre-trade risk checker (cash sufficiency including fees and slippage, short-sell blocking, and a projected max-position-percent bound), and weighted-average-cost position accounting. A `create_replay` path re-binds the identical frozen snapshot and re-runs the engine, and a verification routine proves the stored result matches a fresh recomputation.

The typed policies themselves are textbook constructions: a time-series-momentum rule after Moskowitz et al. [27] (enter when trailing return over the lookback window exceeds an entry threshold, exit below an exit threshold) and a fast/slow SMA crossover (target the position when the fast average is above the slow average). Both are expressed as frozen Pydantic models under a discriminated union with cross-field validators, and the executor derives a long-only target-position transition by comparing the policy's target against the current holding with a floating-point tolerance, so a negligible drift never triggers spurious churn. The design anticipates integration of a formulaic-alpha library [28] as additional deterministic signal generators. An **experimental** agent-driven mode exists but is explicitly labelled non-credible for performance claims (non-goal N1): it runs a scoped `AgentLoop` restricted to `get_price`, `get_indicators`, and a single `submit_backtest_decision` call, or a bounded one-shot structured decision from a JSON-schema-constrained provider, with typed retry/failure categories rather than silent holds.

5.7 Governance, Risk, and the Human-in-the-Loop

Risk analytics computes, from persisted decision-target exposures, a portfolio Value-at-Risk at the 95% and 99% levels as the corresponding historical return quantiles, Conditional VaR (the expected shortfall of the 5% tail, $\mathrm{CVaR}_{95} = \mathbb{E}[r \mid r \le \mathrm{VaR}_{95}]$), maximum drawdown, a pairwise Pearson correlation matrix, and Herfindahl concentration $H = \sum_i w_i^2$ reported both by ticker and by sector, plus a uniform-market-shock stress test whose impact is the shock scaled by gross exposure and which is also anchored to the empirical worst historical day. The service requires at least sixty common return observations before reporting, degrading to an explicit `partial` status otherwise, so a thin history never masquerades as a confident risk estimate.

When a portfolio-manager decision crosses configured thresholds — a position change over 20%, confidence below 0.5, or single-ticker concentration over 30% — a human-in-the-loop rule engine flags it, and an approval **state machine** routes it for human review. The machine admits transitions only from the non-terminal `pending` state to `approved`, `rejected`, `modified`, or `timed_out`; all decision states are terminal and illegal transitions raise an explicit error, so the approval lifecycle cannot be corrupted by out-of-order events. On a `modify` decision the reviewer's amended action and target position replace the originals before execution. This echoes the human-in-the-loop alpha discovery of Alpha-GPT 2.0

[25]. Approvals, decisions, and every tool call are written to the per-strategy audit-trail `events` table, satisfying goal G4 and the accountability requirements of [8].

5.8 Screening, Beliefs, and Knowledge

Three further subsystems support the analytical workflow. The **scanner** screens a tracked universe (tickers unioned from strategies, watchlists, and the market store) against typed conditions using six operators (`< > <= >= == between`) over provenance-tagged snapshot features (price, change, volume, RSI, SMAs, PE, PB, market cap, sector). It offers three entry paths: a purely deterministic **rule** mode; an **agent** mode in which a restricted `AgentLoop` compiles a natural-language query into frozen conditions by calling the scanner tool exactly once; and a **belief** mode that derives conditions from a strategy's stated trading philosophy. The **belief engine** models a trading philosophy as a typed `TradingBelief` (natural-language text, style, tags, weight, and rolling win-rate/return statistics), seeded from five presets — event-driven, conservative-value, momentum-growth, technical-reversal, and macro-sensitive — echoing the conceptual verbal reinforcement of FinCon [5]. The **knowledge base** maintains, per strategy, three Markdown ledgers — verified `rules`, empirical `findings`, and falsified `failures` — that are injected as context before analysis, operationalising the failure-memory idea of Reflexion [20] in a human-auditable form.

5.9 The Skill System and Interoperability

Rather than hard-coding domain methodology into prompts, the system externalises it into seventy-six declarative **skill documents** under nine categories (analysis, strategy, tool, asset-class, flow, crypto, data-sources, research, risk). Each skill is a Markdown file with a YAML frontmatter (`name`, `version`, `category`, `description`, `tools`, `model`, `temperature`) and a methodology body, discovered by a three-layer loader (user overrides → bundled → vendor) and surfaced to the agent through `list_skills` / `search_skills` / `load_skill`. Because user skills override bundled ones by name, a user can specialise the system's behaviour without editing code — a lightweight analogue of tool-learning [19] in which capability is authored declaratively. An MCP [22] base-tool abstraction and client manager provide interoperability scaffolding for exposing tools to, and loading tools from, external agents (currently an interface awaiting a `fastmcp` binding).

5.10 Frontend and Automation

The presentation layer is a Next.js 16 / React 19 application of nineteen routes (dashboard, agent terminal, quick-ask, strategies, backtest, memory lab, insights, approvals, scanner, watchlist, monitor, risk, reports, settings, and a landing page), using TanStack Query for server state, Zustand for minimal UI state, `shadcn/ui` over Tailwind CSS v4, and both TradingView Lightweight Charts and Recharts for visualisation. Beyond the conventional dashboard, the frontend makes several deliberate contributions to human–machine interaction that make the agents' reasoning legible and controllable.

Streaming reasoning transparency. The agent terminal implements a custom Server-Sent Events reader that renders the model's output in three visually distinct tiers: intermediate reasoning in muted grey, tool-call cards appearing in real time (each carrying a clickable

data-source link and, where available, an inline price chart), and the final answer in normal weight. This mirrors the "thinking / answer" split familiar from modern chat products, so the user can follow **why** the agent reached a conclusion rather than only the conclusion itself.

Interleaved event timeline. Rather than grouping all reasoning text and all tool calls into separate blocks, the frontend renders a segments timeline in which each thinking passage is immediately followed by the tool calls it triggered. The user thus sees the exact chronological sequence — "I'll fetch the price" → an `get_price` card → "RSI is overbought" → an `get_indicators` card — preserving the causal order of the agent's process rather than hiding it behind a summary.

Live agent progress with per-node status. On the structured analysis page (Quick Ask), every agent node in the pipeline is shown as a status card whose state advances through waiting → started → completed. A `started` event is emitted from inside the graph node **before** the LLM call begins, via an `on_node_start` callback marshalled through a thread-safe queue, so the blue pulse animation reflects genuinely in-flight work rather than a post-hoc state. This real-time feedback matters most for the multi-minute deep mode, where the user can watch which analyst is currently executing. Every report card, the bull/bear debate history, the three-way risk discussion, and the news-source list are independently collapsible, letting the user drill into exactly as much of the pipeline as they wish.

History ergonomics. Session transcripts are capped at the most recent thirty messages, and the history-list endpoint reads only summary fields rather than parsing full snapshots, keeping navigation fast even after hundreds of analysis runs.

In-process automation is provided by APScheduler: a `DataCollector` refreshes prices (15 min), news (30 min), sentiment (60 min), the macro calendar (daily), and an LLM-driven ticker-discovery pool (every 4 h); a `MonitorRunner` executes user monitors; and a `MorningBriefRunner` generates a daily market brief on weekday mornings.

5.11 A Worked Example: End-to-End Interactive Analysis

To make the interaction between subsystems concrete, consider the request "Should I be worried about NVDA after today's news?" issued to the agent terminal. The server spawns an `AgentLoop` in a worker thread and streams events over SSE. On the first iteration the loop builds the seven-section system prompt, injects the sentiment- and news-analysis skills selected by keyword overlap, and calls the model. The model emits three read-only tool calls — `get_price`, `get_indicators`, and `get_news` for `NVDA` — whose arguments the streaming executor dispatches in parallel the instant each JSON payload completes; `thinking_delta` and `tool_call` events surface live in the browser. Each tool resolves through `DataService`: prices and indicators hit the SHA-256-verified cache (sub-20 ms), while news triggers the Google → AkShare → Yahoo fallback chain and returns article objects with source URLs.

On the second iteration the preprocessing pass finds the transcript well within the 28 K-token warn threshold, so only the zero-cost L0/L1 layers run. The model, now holding grounded data, produces a final answer. Before it is emitted, the answer validator extracts the numeric tokens in the answer (price levels, RSI, percentage moves) and confirms each appears in a

tool result; the grounded answer is returned with a `done` event carrying iteration count, tool count, and elapsed time, and the full transcript is persisted for continuation. Had a provider failed, the recovery ladder would have absorbed the error — a rate-limit would back off and retry, a context overflow would trigger `collapse_drain` — and had the model looped on a repeated `get_price` call, the de-duplication key would have dropped it with a reminder to answer from data already gathered. This single trace exercises the prompt architecture, streaming concurrency, the data plane, compression, recovery, and grounding validation in concert.

The sequence diagram in Figure 3 traces this interaction across the participating components.

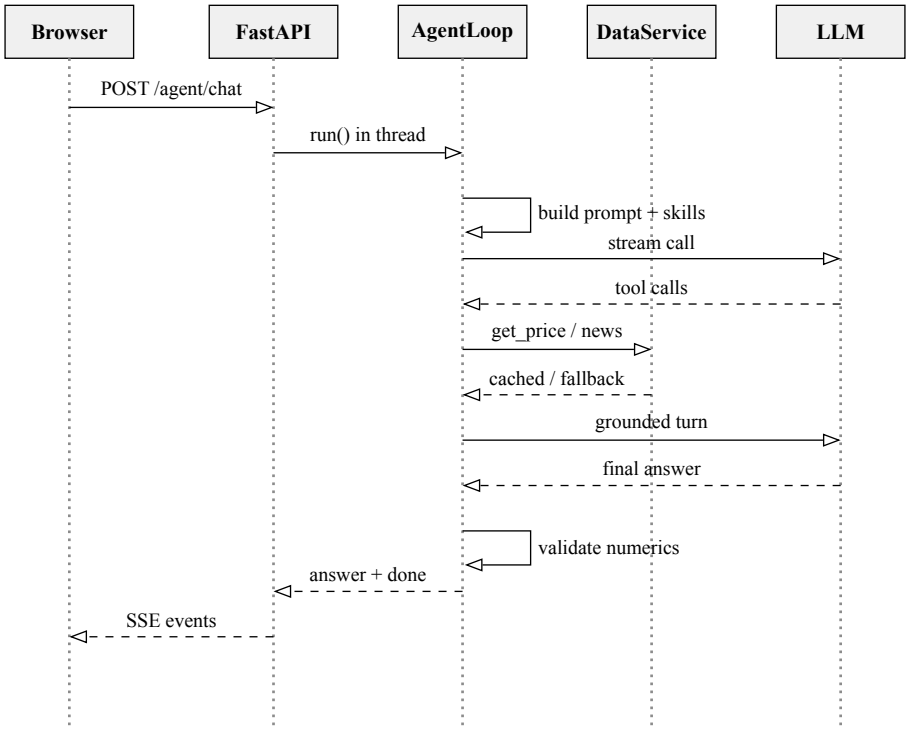

Figure 3: Interactive analysis (Pattern A): the ReAct loop streams reasoning and tool events over SSE, grounding every figure through `DataService` before the answer is validated and returned. Dashed arrows denote returns/streamed responses.

6 Experimental Results and Analysis

Consistent with the design philosophy (non-goal N1), the evaluation does **not** headline a backtest return. Instead it measures the properties the system was actually engineered to guarantee: determinism, latency, grounding, governance, and engineering quality. All figures are drawn from the project's own verification records and regression suite on the `dev` branch.

The evaluation is organised around the six design goals of Section 3; the table below maps each goal to its verification method and headline evidence, and the subsections that follow elaborate.

Goal	Verification method	Headline evidence
G1 Grounding	Tool-output structure tests + answer-validator tests	Un-retrieved figures cannot be emitted without a warning
G2 Reproducibility	Hash-mutation + replay + snapshot-decode tests	Every economic-field mutation fails the read; replay is identical
G3 Point-in-time	`as_of` boundary tests + warm-up isolation tests	No record after the boundary; `insufficient_history` on thin data
G4 Governability	HITL rule + state-machine + approval-route tests	Illegal transitions rejected; every action audited
G5 Learning	OWM scoring + recall re-ranking + safety-gate tests	Context-sensitive recall; five behavioural gates enforced
G6 Maintainability	Architecture import-boundary tests + static analysis	Track boundaries machine-checked; zero diagnostics

Table 12: Mapping of design goals to verification methods and headline evidence.

6.1 Experimental Setup

Experiments use DeepSeek OpenAI-compatible models (`deepseek-v4-flash` for quick reasoning, temperature 0.0) behind the custom-harness loop and the LangGraph pipeline. Data providers are Yahoo Finance, AkShare, Google News, and Finnhub. The regression suite is executed with `pytest`; static analysis uses Ruff and BasedPyright; the frontend is validated with TypeScript, ESLint, and Node tests. The accumulated data plane at the time of evaluation held on the order of thousands of OHLCV rows, news articles, and fundamentals snapshots across a multi-market ticker universe (US, Hong Kong, A-share, Japan, Korea).

6.2 Reproducibility of Backtests (G2)

The central claim — that a persisted backtest is bit-for-bit reproducible — is validated by construction and by adversarial test. A production-created deterministic job over 61 daily bars produced 60 eligible daily decisions and 23 executed orders, with **every** execution occurring on the immediately following historical session after its signal, confirming the next-open rule. Dedicated tests verify that (i) mutating any economic field of a stored result causes the recomputed canonical result hash to diverge and the read to fail; (ii) a `create_replay` run re-binding the frozen snapshot reproduces the identical result; and (iii) snapshot decode integrity is preserved across gzip round-trips and rejects tampering. These constitute direct evidence that reproducibility is enforced rather than assumed — a property absent from the return-centric evaluations in the surveyed literature [2].

6.3 Point-in-Time Safety (G3)

Look-ahead safety is verified at two layers. In the data plane, fundamentals and news queries are shown to return only records dated on or before the `as_of` boundary. In the backtest runner, warm-up bars are selected exclusively from data prior to the evaluation window and excluded from performance, and interior benchmark gaps are forward-filled only up to a bounded staleness before a warning is emitted. Runs lacking sufficient history terminate as typed `insufficient_history` results, so the system fails loudly rather than fabricating a decision.

6.4 Grounding and Anti-Hallucination (G1)

Grounding is defended at three independent layers (data plane, prompt discipline, post-hoc numeric validation) and asserted by tests on tool output structure and on the answer validator. Because data functions return explicit empty/error envelopes on provider failure, the agent is structurally prevented from emitting an un-retrieved figure without triggering either an "Unknown" annotation or a validator warning. This directly addresses the hallucination failure mode identified as a production risk by [2] and [12].

6.5 Latency and Cost Profile

Warm data reads served from the SHA-256-verified cache complete in under about 20 ms, whereas cold provider fetches take on the order of hundreds of milliseconds; a full multi-agent LangGraph analysis and a real deterministic backtest job complete in tens of seconds (a representative end-to-end job finished in roughly 55 seconds). The compression pipeline keeps interactive sessions within a bounded token budget across many tool calls, and the recovery ladder absorbs transient provider failures without aborting the session. These properties matter because the evaluation desiderata of [2] explicitly require reporting latency and cost, not only accuracy.

6.6 Engineering Quality

The system sustains a substantial automated-verification regime: a regression suite of **645 tests** passing (with one skip), zero Ruff and BasedPyright diagnostics, and a passing production frontend build across twenty pages, as recorded in the project's progress log. The test suite spans 59 files and roughly 21,000 lines, covering the agent loop, tool allow-listing, backtest determinism and hash mutations, storage migration integrity, snapshot decode integrity, risk and scanner services, and the HITL flow. Automated architecture tests enforce the track import boundaries (goal G6), converting a design rule into a machine-checked invariant.

Quality dimension	Result
Regression tests	645 passing / 1 skipped
Test corpus	59 files, 21 K LoC
Static analysis (Ruff, BasedPyright)	0 diagnostics
Backtest determinism (hash-mutation tests)	enforced
Import-boundary architecture tests	enforced
Frontend build / TypeScript / Node tests	passing

Table 13: Engineering-quality dimensions and results.

6.7 Summary of Findings

The evaluation supports the dissertation's thesis: the system reliably delivers the **verifiability** properties it was designed for. Reproducibility, point-in-time safety, and grounding are not merely claimed but enforced by hashes, typed contracts, and tests. The cost of this rigour is that the system makes no validated alpha claim — a deliberate and, we argue, honest trade-off.

7 Discussion

7.1 Interpretation

The results reframe what "success" means for an LLM trading agent. The surveyed systems [4], [5], [17] report impressive returns, but the same survey literature [2] that catalogues these results also enumerates the methodological hazards that make many of them hard to trust. Agentic-Quant demonstrates that it is possible — and not prohibitively expensive — to build the missing substrate: a system in which the analytical machinery is flexible and LLM-driven, yet every downstream artefact is grounded, reproducible, and governed. In effect, the project trades a headline number for an **auditable process**, which is the more defensible deliverable in a regulated domain [8].

7.2 Design Trade-offs

Three trade-offs merit explicit comment. First, the **dual-track** decision doubles some conceptual surface area (two orchestration styles, two state representations) in exchange for isolating experimental work from stable code; the enforced import boundary and the single call-time bridge keep this cost contained. Second, the **deterministic backtest** forgoes per-bar LLM reasoning — arguably the most "intelligent" mode — precisely because such reasoning is non-reproducible; the system relegates it to an explicitly non-credible experimental mode rather than deleting it. Third, the **monolithic single-process** design (non-goal N3) sacrifices horizontal scalability for debuggability, an appropriate choice for a research prototype but one that would need revisiting for production.

7.3 Limitations

The system has clear boundaries. The MCP interoperability layer is scaffolding pending a `fastmcp` integration; the HITL executor currently logs rather than dispatching to the broker; the belief-contest weighting is implemented as configuration rather than a fully closed evaluation loop; and the historical LangGraph `agents/` and `agentgraph/` directories have been archived into `quick_ask/legacy/` since the `quick_ask/` variant is the maintained LangGraph pipeline. Sentiment analysis is keyword-based rather than model-based, a deliberate simplicity/cost choice that a finance-tuned classifier [14] could improve. Most importantly, the deterministic policies are simple technical rules; the framework is a **platform** for grounded analysis, not a validated alpha strategy.

7.4 Threats to Validity

The latency and data-volume figures are drawn from a development environment with rate-limited public data sources and therefore indicate order-of-magnitude behaviour rather than production SLAs. Grounding validation catches numeric fabrication but cannot detect subtly wrong **reasoning** over correct numbers. And because non-goal N1 forbids alpha claims, the evaluation cannot and does not speak to economic performance; readers should not infer profitability from the reproducibility results.

8 Conclusion and Future Work

8.1 Conclusion

This dissertation presented Agentic-Quant, a reasoning-driven multi-agent framework for quantitative financial analysis that deliberately privileges verifiability over headline return. Its principal contributions are a dual-track agent architecture that hosts both a modern ReAct tool-use loop and a frozen LangGraph debate pipeline; an anti-hallucination data plane with point-in-time discipline and verified caching; a five-factor outcome-weighted memory with behavioural safety gates; and a deterministic, hash-audited backtest engine that makes reproducibility an enforced invariant. Across roughly 54,000 lines of Python, a sixteen-route React dashboard, seventy-six declarative skills, and a 645-test regression suite, the system demonstrates that an LLM-driven trading agent can be engineered as a **governed, reproducible research instrument**. We argue this reframing — from black-box oracle to auditable process — is a necessary precondition for the responsible use of LLM agents in finance, and that the engineering substrate developed here is a reusable answer to the reproducibility and grounding gaps repeatedly flagged in the literature [2], [8].

8.2 Future Work

Several directions follow naturally. **Closing the belief-contest loop** would let multiple trading philosophies [5] compete on recorded performance and re-weight automatically. **Integrating a formulaic-alpha library** [28] would provide a rich set of deterministic signals within the reproducibility contract. **Model-based sentiment and event extraction** [14], [15] would replace the keyword aggregator. **Wiring the HITL executor to the broker**

and **completing the MCP binding** [22] would close the governance and interoperability loops. Finally, a **separately specified out-of-sample research protocol** — walk-forward evaluation with cost, latency, and capacity accounting as mandated by [2] — would allow the platform to make, for the first time and on defensible terms, an actual performance claim.

References

[1] N. Pippas, E. A. Ludvig, and C. Turkay, "The Evolution of Reinforcement Learning in Quantitative Finance: A Survey," *ACM Computing Surveys*, vol. 57, no. 11, pp. 1–51, Nov. 2025, doi: 10.1145/3733714.

[2] W. Fu, "The New Quant: A Survey of Large Language Models in Financial Prediction and Trading." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2510.05533

[3] Y. Dong *et al.*, "Large Language Model Agents in Finance: A Survey Bridging Research, Practice, and Real-World Deployment," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, Suzhou, China: Association for Computational Linguistics, 2025, pp. 17889–17907. doi: 10.18653/v1/2025.findings-emnlp.972.

[4] Y. Xiao, E. Sun, D. Luo, and W. Wang, "TradingAgents: Multi-Agents LLM Financial Trading Framework." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2412.20138

[5] Y. Yu *et al.*, "FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2407.06567

[6] Y. Yu *et al.*, "FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2311.13743

[7] H. Yang *et al.*, "FinRobot: An Open-Source AI Agent Platform for Financial Applications using Large Language Models." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2405.14767

[8] H. Tatsat and A. Shater, "Beyond the Black Box: Interpretability of LLMs in Finance." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2505.24650

[9] S. Wu *et al.*, "BloombergGPT: A Large Language Model for Finance." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2303.17564

[10] H. Yang, X.-Y. Liu, and C. D. Wang, "FinGPT: Open-Source Financial Large Language Models." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2306.06031

[11] G. Fatouros, K. Metaxas, J. Soldatos, and D. Kyriazis, "Can Large Language Models beat wall street? Evaluating GPT-4's impact on financial decision-making with MarketSenseAI," *Neural Computing and Applications*, vol. 37, no. 30, pp. 24893–24918, Oct. 2025, doi: 10.1007/s00521-024-10613-4.

[12] X. Li *et al.*, "AlphaFin: Benchmarking Financial Analysis with Retrieval-Augmented Stock-Chain Framework." Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2403.12582

[13] K. Kirtac and G. Germano, “Enhanced Financial Sentiment Analysis and Trading Strategy Development Using Large Language Models,” in *Proceedings of the 14th Workshop on Computational Approaches to Subjectivity, Sentiment, & Social Media Analysis*, Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 1–10. doi: 10.18653/v1/2024.wassa-1.1.

[14] J. Delgadillo, J. Kinyua, and C. Mutigwe, “FinSoSent: Advancing Financial Market Sentiment Analysis through Pretrained Large Language Models,” *Big Data and Cognitive Computing*, vol. 8, no. 8, p. 87, Aug. 2024, doi: 10.3390/bdcc8080087.

[15] M. Wang, S. B. Cohen, and T. Ma, “Modeling News Interactions and Influence for Financial Market Prediction,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 3302–3314. doi: 10.18653/v1/2024.findings-emnlp.189.

[16] W. Zhang *et al.*, “A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Barcelona Spain: ACM, Aug. 2024, pp. 4314–4325. doi: 10.1145/3637528.3671801.

[17] F. Tian, F. D. Salim, and H. Xue, “TradingGroup: A Multi-Agent Trading System with Self-Reflection and Data-Synthesis.” Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2508.17565

[18] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models.” [Online]. Available: http://arxiv.org/abs/2210.03629

[19] T. Schick *et al.*, “Toolformer: Language Models Can Teach Themselves to Use Tools.” [Online]. Available: http://arxiv.org/abs/2302.04761

[20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning.” [Online]. Available: http://arxiv.org/abs/2303.11366

[21] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” [Online]. Available: http://arxiv.org/abs/2005.11401

[22] Anthropic, “Introducing the Model Context Protocol.” [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[23] Q. Zhang *et al.*, “Unifying Language Agent Algorithms with Graph-based Orchestration Engine for Reproducible Agent Research,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 107–117. doi: 10.18653/v1/2025.acl-demo.11.

[24] A. de-la-Rica-Escudero, E. C. Garrido-Merchán, and M. Coronado-Vaca, “Explainable post hoc portfolio management financial policy of a Deep Reinforcement Learning agent,” *PLOS ONE*, vol. 20, no. 1, p. e315528, Jan. 2025, doi: 10.1371/journal.pone.0315528.

[25] H. Yuan, S. Wang, and J. Guo, “Alpha-GPT 2.0: Human-in-the-Loop AI for Quantitative Investment.” Accessed: Mar. 25, 2026. [Online]. Available: http://arxiv.org/abs/2402.09746

[26] W. F. Sharpe, “The Sharpe Ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994, doi: 10.3905/jpm.1994.409501.

[27] T. J. Moskowitz, Y. H. Ooi, and L. H. Pedersen, “Time series momentum,” *Journal of Financial Economics*, vol. 104, no. 2, pp. 228–250, 2012, doi: 10.1016/j.jfineco.2011.11.003.

[28] Z. Kakushadze, “101 Formulaic Alphas.” [Online]. Available: http://arxiv.org/abs/1601.00991

Appendix A: Persistence Layout

The table below records the physical database layout that realises the per-strategy isolation principle of Section 5.5. Runtime databases are git-ignored and rebuilt from public sources; only schema-defining code is version-controlled.

Database	Contents
`system.db`	Strategy registry, watchlists, monitors, scan runs, report jobs, insight generations, backtest jobs and snapshots
`market_data.db`	OHLCV (compound key), fundamentals (point-in-time), news + FTS5 index, ticker metadata, data-freshness tracking
`memory.db`	OWM memory records with per-strategy and score indexes
`insights.db`	Daily briefs and monitoring reports
`data/{strategy_id}.db`	Per-strategy sessions, agent reports, decisions, events (audit trail), HITL approvals
`data/agent-runs/`	ReAct session transcripts and JSONL traces with blob off-loading
`skills/` (filesystem)	76 declarative SKILL.md documents plus per-strategy Markdown knowledge ledgers

Table 14: Physical database layout (Appendix A).

Appendix B: Reproducibility Chain

For completeness, the chain of hashes and typed contracts that together guarantee a reproducible backtest (Section 5.6) is: a frozen `BacktestRunSpec` pinning dates, mode, typed policy, and broker configuration; a `policy_hash` and `strategy_snapshot_hash` over canonical JSON; a `gzip`-compressed input snapshot whose `content_hash` is verified on write and re-verified on read; per-decision evidence rows carrying a `feature_hash` and `policy_hash`; and a canonical **economic result hash** recomputed on every read, with any mismatch downgrading the job to `storage_corrupt`. The `create_replay` path re-binds the identical snapshot and re-runs the deterministic engine, providing an independent reproduction of the persisted result.